Werkzeuge · Warm-up-Karten

Warm-up-Karten Welt 3 – Das Labyrinth

Kurze Papier-Aufgaben für den Einstieg jeder Doppelstunde („10' Papier-Warm-up, ohne Gerät"). Jede Karte ist in 5–8 Minuten handschriftlich lösbar. Die Karten W3-01 bis W3-14 begleiten das Labyrinth (DS 18–22): Rekursion (Rekursionstabellen mit Spalte Rekursionstiefe, Aufrufe zählen, fehlender Basisfall, Rekursion statt Schleife), die Dungeon-Karte als 2D-Reihung (Index, Rand, Nachbarn, Flood-Fill), Backtracking und Teile und herrsche (binäre Suche, Mergesort, Quicksort) bis zur Analyse unbekannter rekursiver Verfahren.

So funktioniert's

- Karte ziehen (oder zugeteilt bekommen), **ohne Gerät** auf einem eigenen Blatt lösen.
- Selbst- oder Partnerkontrolle mit dem Lösungsteil (gleiche Nummer). Fehler ins Fehlerlogbuch – Warm-up-Karten sind Training ohne Punkte.
- Digital im **Warm-up-Trainer** des Algo Quest Lab (Menü „Warm-up", Welt-Filter „Welt 3"): erst auf Papier lösen, dann aufdecken und ehrlich einschätzen (Leitner-System, ohne Punkte).
- Boss-Training vor Boss 3: Karten W3-01, W3-07, W3-11, W3-13 und W3-14. Zusätzlich im Lab: Online-Boss **„Das Echo der Schlucht"**.

Achtung Abi:

Im Abitur schreibst du alles von Hand. Rekursionsverläufe dokumentierst du in einer Tabelle mit der Spalte **Rekursionstiefe** (Erstaufruf = Tiefe 0, jeder Selbstaufruf + 1) – notiere jeden Aufruf in der Reihenfolge, in der er beginnt. Prüfe jede rekursive Methode auf Basisfall und Verkleinerung des Problems (sonst Endlosrekursion), 2D-Zugriffe immer erst nach der Bereichsprüfung (Kurzschluss mit „und"). Effizienz wird über das Zählen von Aufrufen und Vergleichen und über Verdopplung beurteilt.

Kartenübersicht

Nr.	Kategorie	Titel	Min.	Einsatz
W3-01	Tracetabelle	Das Echo im Brunnen	6	DS 18 + Boss
W3-02	Funktionsweise erläutern	Die Hydra des Labyrinths	7	DS 18
W3-03	Fehler im Java-Code	Der endlose Abstieg	7	DS 18
W3-04	Methodenkopf und Signatur	Zählen ohne Schleife	7	DS 18
W3-05	Lückencode	Koordinaten im Dungeon	6	DS 19
W3-06	Methode implementieren	Der Fallen-Sensor	7	DS 19
W3-07	Struktogramm lesen	Den Nebel lüften	7	DS 19 + Boss
W3-08	Tracetabelle	Der Weg durchs Labyrinth	8	DS 20 (Kür)
W3-09	Tracetabelle	Die Schatzkammer-Suche	7	DS 21
W3-10	Funktionsweise erläutern	Der Spaltzauber	7	DS 21
W3-11	Code ↔ Struktogramm	Der stärkste Golem	7	DS 21 + Boss
W3-12	Tracetabelle	Die Pivot-Wache	8	DS 22
W3-13	Fehler im Struktogramm	Das falsche Echo	6	DS 22 + Boss
W3-14	Funktionsweise erläutern	Das Rätsel des Turmwächters	7	DS 22 + Boss

Druckhinweis: Kartenseiten einseitig drucken, an den gestrichelten Linien schneiden (2 Karten im Format A5 je Seite), ggf. laminieren. Der Lösungsteil ist für die Kontrollstation bzw. das Lehrerexemplar gedacht.

WARM-UP W3-01 Tracetabelle

Welt 3 – Labyrinth · ⏱ 6 min

Das Echo im Brunnen

Ruft man in den Brunnen des Labyrinths eine Zahl, antwortet das Echo mit der Methode echo.

a) Dokumentiere den Aufruf echo(4718) in einer Rekursionstabelle mit den Spalten Rekursionstiefe | Aufruf | Rückgabewert. Rekursionstiefe: Der Erstaufruf hat die Tiefe 0, jeder rekursive Aufruf erhöht die Tiefe um 1.

b) Beschreibe, was echo(n) für n ≥ 0 allgemein berechnet.

c) Kennzeichne Basisfall und Rekursionsschritt und begründe, dass die Rekursion für jedes n ≥ 0 endet.

```
public static int echo(int n) {
    if (n < 10) {
        return n;
    }
    return echo(n / 10) + n % 10;
}
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W3-01

WARM-UP W3-02 Funktionsweise erläutern

Welt 3 – Labyrinth · ⏱ 7 min

Die Hydra des Labyrinths

Für jeden abgeschlagenen Kopf wachsen der Hydra neue nach – die Methode koepfe beschreibt das.

a) Zeichne den Aufrufbaum für koepfe(4) und gib den Rückgabewert an.

b) Gib die Anzahl aller Aufrufe (einschließlich des Erstaufrufs) für koepfe(4) und koepfe(5) an. Wie oft wird bei koepfe(5) der Aufruf koepfe(2) ausgeführt?

c) Für die Anzahl A(n) der Aufrufe gilt A(0) = A(1) = 1. Stelle eine Formel für A(n) bei n ≥ 2 auf und schätze ab, wie sich A(n) verändert, wenn n um 1 wächst. Was bedeutet das für koepfe(40)?

```
public static int koepfe(int n) {
    if (n <= 1) {
        return 1;
    }
    return koepfe(n - 1) + koepfe(n - 2);
}
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W3-02

WARM-UP W3-03 Fehler im Java-Code

Welt 3 – Labyrinth · ⏱ 7 min

Der endlose Abstieg

Jede Methode enthält einen Fehler, der zu einer Endlosrekursion führen kann – Java bricht dann mit einem StackOverflowError ab (der Aufrufstapel läuft über). Es gilt jeweils n ≥ 0 bzw. exponent ≥ 0.

a) Beschreibe jeweils den Fehler und was bei summeBis(3), potenz(2, 3) und jedeZweite(7) geschieht.

b) Gib je Methode einen Aufruf an, der trotz des Fehlers korrekt endet – oder begründe, dass es keinen gibt.

c) Korrigiere die Methoden und nenne die zwei Bedingungen, die jede Rekursion erfüllen muss.

```
// soll 0 + 1 + 2 + ... + n liefern
public static int summeBis(int n) {
    return n + summeBis(n - 1);
}
// soll basis hoch exponent liefern
public static int potenz(int basis, int exponent) {
    if (exponent == 0) {
        return 1;
    }
    return basis * potenz(basis, exponent);
}
// soll n + (n - 2) + (n - 4) + ... liefern, z. B. 7 + 5 + 3 + 1 = 16
public static int jedeZweite(int n) {
    if (n == 0) {
        return 0;
    }
    return n + jedeZweite(n - 2);
}
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W3-03

WARM-UP W3-04 Methodenkopf und Signatur

Welt 3 – Labyrinth · ⏱ 7 min

Zählen ohne Schleife

Im Labyrinth wirkt ein Bann: Schleifen sind verboten. Die Methode zaehle(s: Zeichenkette, c: Zeichen): Ganzzahl zählt, wie oft das Zeichen c in s vorkommt – bisher mit einer Schleife. Die rekursive Fassung braucht eine Hilfsoperation zaehleAb(s: Zeichenkette, c: Zeichen, i: Ganzzahl): Ganzzahl, die die Vorkommen von c ab dem Index i zählt.

a) Gib den Java-Methodenkopf von zaehleAb an. Gib an, wie zaehle die Hilfsoperation startet (Rumpf von zaehle).

b) Implementiere zaehleAb rekursiv ohne Schleife (Basisfall: i == s.length()).

c) Gib zum Java-Kopf public static boolean enthaeltAb(int[] r, int x, int i) die Signatur in Abi-Notation an. Erläutere, was in der rekursiven Fassung an die Stelle der Schleifenvariablen i und der Variablen anzahl tritt.

```
public static int zaehle(String s, char c) {
    int anzahl = 0;
    for (int i = 0; i < s.length(); i++) {
        if (s.charAt(i) == c) {
            anzahl++;
        }
    }
    return anzahl;
}
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W3-04

WARM-UP W3-05 Lückencode

Welt 3 – Labyrinth · ⏱ 6 min

Koordinaten im Dungeon

Die Dungeon-Karte ist eine 2D-Reihung feld von Zeichen; feld[z][s] ist das Feld in Zeile z und Spalte s. Unten siehst du die Karte „Höhle“ aus dem Kartenbuch der Online-IDE.

a) Gib an: feld.length, feld[0].length, feld[1][1], feld[3][5], feld[2][3]. Was geschieht bei feld[5][0] und bei feld[1][7]?

b) Ergänze die Lücken (1) bis (5) der Methoden istImFeld und anzahlFrei (zählt die Felder mit '.').

c) Gib den Rückgabewert von anzahlFrei() an und erläutere, warum in istImFeld(z, s) && feld[z][s] != '#' die Reihenfolge der beiden Bedingungen wichtig ist.

```
public boolean istImFeld(int z, int s) {
    return z >= 0 && z < ___(1)___ && s >= 0 && s < ___(2)___;
}

public int anzahlFrei() {
    int anzahl = 0;
    for (int z = 0; z < ___(3)___; z++) {
        for (int s = 0; s < ___(4)___; s++) {
            if (___(5)___ == '.') {
                anzahl++;
            }
        }
    }
    return anzahl;
}
```

```
    s:  0 1 2 3 4 5 6
z = 0:  # # # # # # #
z = 1:  # S . . . . #
z = 2:  # . . # . . #
z = 3:  # . . . . Z #
z = 4:  # # # # # # #
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W3-05

WARM-UP W3-06 Methode implementieren

Welt 3 – Labyrinth · ⏱ 7 min

Der Fallen-Sensor

Ein Fallen-Sensor zählt die Wände ('#') in der 8er-Nachbarschaft eines Feldes – also auch diagonal. Felder außerhalb der Karte zählen nicht. In der Klasse Karte gibt es bereits istImFeld(z: Ganzzahl, s: Ganzzahl): Wahrheitswert und getFeld(z: Ganzzahl, s: Ganzzahl): Zeichen.

a) Gib die Koordinaten (z|s) aller acht Nachbarn von (1|1) an.

b) Implementiere die Methode wandNachbarn8(z: Ganzzahl, s: Ganzzahl): Ganzzahl in Java. Tipp: zwei Zählschleifen für die Verschiebungen dz und ds von −1 bis 1.

c) Gib für die Karte „Höhle“ unten die Rückgabewerte für (1|1), (0|0) und (2|2) an.

```
    s:  0 1 2 3 4 5 6
z = 0:  # # # # # # #
z = 1:  # S . . . . #
z = 2:  # . . # . . #
z = 3:  # . . . . Z #
z = 4:  # # # # # # #
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W3-06

WARM-UP W3-07 Struktogramm lesen

Welt 3 – Labyrinth · ⏱ 7 min

Den Nebel lüften

Die „Lichtung“ aus dem Kartenbuch ist noch vernebelt und hat am Rand KEINE Wände. lueften markiert alle vom Startfeld aus erreichbaren begehbaren Felder; istBegehbar(z, s) prüft zuerst, ob (z|s) in der Karte liegt, und dann, ob dort '.', 'S' oder 'Z' steht. Am Ende sind alle erreichten '.' gelüftet ('+'), S und Z bleiben sichtbar.

a) Führe lueften(1, 1) aus. Nummeriere die Felder (auch S und Z) in der Reihenfolge, in der sie auf '+' gesetzt werden, und gib jeweils die Rekursionstiefe an. Rekursionstiefe: Der Erstaufruf hat die Tiefe 0, jeder rekursive Aufruf erhöht die Tiefe um 1.

b) Gib die Anzahl aller Aufrufe von lueften an – auch derer, die sofort enden – und begründe sie ohne Nachzählen.

c) Erläutere, was passiert, wenn feld[z][s] ← '+' erst nach den vier Selbstaufrufen ausgeführt wird, und warum auch S und Z vorübergehend '+' werden.

```
lueften(z: Ganzzahl, s: Ganzzahl): nichts
  falls istBegehbar(z, s)
    dann: alt ← feld[z][s]
          feld[z][s] ← '+'
          lueften(z − 1, s)      // oben
          lueften(z, s + 1)      // rechts
          lueften(z + 1, s)      // unten
          lueften(z, s − 1)      // links
          falls alt = 'S' oder alt = 'Z'
            dann: feld[z][s] ← alt
    sonst: –
```

```
     s:  0 1 2 3
z = 0:   . . # .
z = 1:   . S . .
z = 2:   # . . Z
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W3-07

WARM-UP W3-08 Tracetabelle

Welt 3 – Labyrinth · ⏱ 8 min

Der Weg durchs Labyrinth

findeWeg sucht per Backtracking einen Weg vom Start zum Ziel 'Z' (wie Level 3.2 Experte). Betretene Felder werden '*'; führt von einem Feld kein Weg zum Ziel, wird der Schritt zurückgenommen ('x' = Sackgasse). „oder“ bricht beim ersten wahr ab. Start: findeWeg(1, 1) auf der Lichtung, schritte = 0.

a) Dokumentiere alle Aufrufe, die ein Feld betreten ('.' oder 'S'), in einer Tabelle Rekursionstiefe | Feld (z|s) | Zeichen am Ende. Rekursionstiefe: Der Erstaufruf hat die Tiefe 0, jeder rekursive Aufruf erhöht die Tiefe um 1.

b) Gib die Karte nach dem Ende und den Wert von schritte an.

c) Erläutere, was „einen Schritt zurücknehmen“ hier bedeutet, und begründe, warum das Feld dabei 'x' und nicht wieder '.' wird.

```
findeWeg(z: Ganzzahl, s: Ganzzahl): Wahrheitswert
  schritte ← schritte + 1
  falls nicht istImFeld(z, s): gib falsch zurück
  alt ← feld[z][s]
  falls alt = 'Z': gib wahr zurück
  falls alt ≠ '.' und alt ≠ 'S': gib falsch zurück
  feld[z][s] ← '*'
  falls findeWeg(z − 1, s) oder findeWeg(z, s + 1)
        oder findeWeg(z + 1, s) oder findeWeg(z, s − 1)
    dann: falls alt = 'S': feld[z][s] ← 'S'
          gib wahr zurück
  feld[z][s] ← 'x'                        // zurücknehmen
  falls alt = 'S': feld[z][s] ← 'S'
  gib falsch zurück
```

```
     s:  0 1 2 3
z = 0:   . . # .
z = 1:   . S . .
z = 2:   # . . Z
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W3-08

WARM-UP W3-09 Tracetabelle

Welt 3 – Labyrinth · ⏱ 7 min

Die Schatzkammer-Suche

Die Schatztruhen der Kammer sind aufsteigend nach Gold sortiert: r = [3, 8, 12, 17, 21, 26, 30, 35, 41, 47] (Indizes 0 bis 9). binaerRek sucht rekursiv nach dem Wert x.

a) Dokumentiere die Aufrufe binaerRek(r, 30, 0, 9) und binaerRek(r, 9, 0, 9) jeweils in einer Tabelle Rekursionstiefe | lo | hi | mid | r[mid] | Rückgabewert. Rekursionstiefe: Der Erstaufruf hat die Tiefe 0, jeder rekursive Aufruf erhöht die Tiefe um 1.

b) Erläutere an der Methode das Prinzip „Teile und herrsche“.

c) Gib an, wie viele Aufrufe bei 10 Elementen höchstens nötig sind (einschließlich eines Aufrufs mit lo > hi), und wie viele zusätzliche Aufrufe eine Verdopplung auf 20 Elemente höchstens kostet.

```
public static int binaerRek(int[] r, int x, int lo, int hi) {
    if (lo > hi) {
        return -1;
    }
    int mid = (lo + hi) / 2;
    if (r[mid] == x) {
        return mid;
    }
    if (r[mid] < x) {
        return binaerRek(r, x, mid + 1, hi);
    }
    return binaerRek(r, x, lo, mid - 1);
}
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W3-09

WARM-UP W3-10 Funktionsweise erläutern

Welt 3 – Labyrinth · ⏱ 7 min

Der Spaltzauber

mergeSort teilt eine Reihung mit mitte = länge / 2 in links = r[0 … mitte − 1] und rechts = r[mitte … länge − 1], sortiert beide Teile rekursiv und verschmilzt sie mit verschmelze. Reihungen der Länge ≤ 1 werden direkt (als Kopie) zurückgegeben.

a) Zeichne für r = [38, 27, 43, 3, 9, 82, 10] den Teilungsbaum bis zu den Reihungen der Länge 1 und darunter den Verschmelzungsbaum.

b) Gib an, wie oft mergeSort (einschließlich des Erstaufrufs) und wie oft verschmelze aufgerufen werden.

c) verschmelze vergleicht nur, solange beide Teile noch Elemente haben. Gib an, wie viele Vergleiche das Verschmelzen von [3, 27, 38] und [9, 10, 43, 82] benötigt.

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W3-10

WARM-UP W3-11 Code ↔ Struktogramm

Welt 3 – Labyrinth · ⏱ 7 min

Der stärkste Golem

maximumTH bestimmt nach dem Prinzip „Teile und herrsche“ den größten Wert in r[von] … r[bis] (von ≤ bis).

a) Übertrage die Methode in ein Struktogramm nach den Abi-Konventionen. Rekursive Aufrufe sind normale Anweisungen, z. B. links ← maximumTH(r, von, mitte).

b) Gib die Anzahl der Aufrufe und die größte Rekursionstiefe für maximumTH(r, 0, 7) an (8 Elemente). Rekursionstiefe: Der Erstaufruf hat die Tiefe 0, jeder rekursive Aufruf erhöht die Tiefe um 1.

c) Ein Golem-Bauer ändert die Mitte zu mitte = (von + bis + 1) / 2 und lässt die Aufrufe unverändert. Erläutere am Aufruf maximumTH(r, 0, 1), was passiert.

```
public static int maximumTH(int[] r, int von, int bis) {
    if (von == bis) {
        return r[von];
    }
    int mitte = (von + bis) / 2;
    int links = maximumTH(r, von, mitte);
    int rechts = maximumTH(r, mitte + 1, bis);
    if (links > rechts) {
        return links;
    }
    return rechts;
}
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W3-11

WARM-UP W3-12 Tracetabelle

Welt 3 – Labyrinth · ⏱ 8 min

Die Pivot-Wache

quickSort(r, lo, hi) ruft für lo < hi zuerst p ← zerlege(r, lo, hi) auf und sortiert danach rekursiv r[lo … p − 1] und r[p + 1 … hi]. Für lo ≥ hi endet der Aufruf sofort. Startaufruf: quickSort(r, 0, 6) mit r = [5, 8, 2, 9, 1, 7, 3].

a) Gib die Belegung von r nach zerlege(r, 0, 6), den Rückgabewert und die Anzahl der Vergleiche r[i] < pivot an.

b) Dokumentiere alle Aufrufe mit lo < hi in einer Tabelle Rekursionstiefe | Aufruf | Rückgabe von zerlege | r nach zerlege. Wie viele Aufrufe von quickSort gibt es insgesamt?

c) Gib die Anzahl der Vergleiche für die bereits sortierte Reihung [1, 2, 3, 4, 5, 6, 7] an und erläutere, warum das der ungünstigste Fall ist.

```
public static int zerlege(int[] r, int lo, int hi) {
    int pivot = r[lo];
    int grenze = lo;
    for (int i = lo + 1; i <= hi; i++) {
        if (r[i] < pivot) {
            grenze = grenze + 1;
            int hilf = r[grenze];
            r[grenze] = r[i];
            r[i] = hilf;
        }
    }
    int hilf = r[lo];
    r[lo] = r[grenze];
    r[grenze] = hilf;
    return grenze;
}
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W3-12

WARM-UP W3-13 Fehler im Struktogramm

Welt 3 – Labyrinth · ⏱ 6 min

Das falsche Echo

anzahlGroesser soll rekursiv zählen, wie viele Werte ab dem Index i größer als grenze sind. Gestartet wird mit i = 0. Das Struktogramm enthält zwei Fehler.

a) Erläutere, was beim Aufruf anzahlGroesser([4, 9, 2, 7], 5, 0) geschieht.

b) Nimm an, nur der Fehler aus a) ist behoben. Gib den Rückgabewert für dieselbe Eingabe an (richtig wäre 2) und erläutere den zweiten Fehler. Was geschieht zusätzlich bei einer leeren Reihung?

c) Gib ein korrigiertes Struktogramm an.

```
anzahlGroesser(r: Reihung von Ganzzahl, grenze: Ganzzahl, i: Ganzzahl): Ganzzahl
  falls i = Länge von r − 1
    dann: gib 0 zurück
    sonst: –
  falls r[i] > grenze
    dann: gib 1 + anzahlGroesser(r, grenze, i + 1) zurück
    sonst: gib anzahlGroesser(r, grenze, i) zurück
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W3-13

WARM-UP W3-14 Funktionsweise erläutern

Welt 3 – Labyrinth · ⏱ 7 min

Das Rätsel des Turmwächters

Der Turmwächter ordnet seine Schlüssel mit einer Methode, deren Namen er geheim hält. Gestartet wird mit raetsel(r, r.length).

a) Dokumentiere den Aufruf raetsel(r, 4) für r = [6, 2, 9, 4] in einer Tabelle Rekursionstiefe | Aufruf | Belegung von r zum Zeitpunkt des Aufrufs. Rekursionstiefe: Der Erstaufruf hat die Tiefe 0, jeder rekursive Aufruf erhöht die Tiefe um 1.

b) Beschreibe, was raetsel bewirkt, und benenne das bekannte Sortierverfahren, das hier rekursiv umgesetzt ist. Was ist der Basisfall, was wird im Rekursionsschritt kleiner?

c) Gib die Anzahl der Vergleiche r[i] > r[m] für n = 4 und allgemein für n Elemente an. Wie verändert sich die Anzahl ungefähr, wenn n verdoppelt wird?

```
public static void raetsel(int[] r, int n) {
    if (n <= 1) {
        return;
    }
    int m = 0;
    for (int i = 1; i < n; i++) {
        if (r[i] > r[m]) {
            m = i;
        }
    }
    int hilf = r[m];
    r[m] = r[n - 1];
    r[n - 1] = hilf;
    raetsel(r, n - 1);
}
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W3-14